

05_負の数・補数・シフト

1. 今回の目標

- ・ 0/1 の並びで負の数も表す必要があることを説明できる
- ・ 2 の補数の作り方を説明できる
- ・ 左シフト・右シフトの基本的な意味を説明できる
- ・ オーバーフローとは何かを説明できる
- ・ 浮動小数点には誤差が出ることがあると理解する

今日の流れ

復習 → 負の数の必要性 → 補数 → 紙ベース実習① → シフト → 紙ベース実習② → オーバーフロー → 浮動小数点 → まとめ → 学習カルテ

2. 前回の要点を確認しよう

- ・ 論理演算は、真と偽を扱う
- ・ ビット演算は、2進数の各桁を1つずつ見る
- ・ NOT は 0 と 1 を反転する
- ・ AND を使うと特定のビットを取り出せる

今日につながる点

今回は、0/1 の並びを使って **負の数** も表す。また、ビットを左右に動かす **シフト** も扱う。

3. 負の数をどう表すか

10 進数では、負の数を -3, -5 のように書ける。

しかし、コンピュータの中では 0 と 1 だけで表す必要がある。

- ・ 正の数だけでなく、負の数も表したい
- ・ そのために、負の数のための表し方が必要になる
- ・ その代表が 2 の補数 である

4. 補数とは何か

補数とは、ある数に足すと区切りのよい値になる数である。

10 進数の例：

- ・ 3 の 10 の補数は 7 ($3 + 7 = 10$)
- ・ 27 の 100 の補数は 73 ($27 + 73 = 100$)

2 進数でも、同じように考えることができる。

5.2 の補数の作り方

4 ビットで 0011 (3) を考える。

1. 0 と 1 を反転する 0011 1100
2. 1 を足す $1100 + 1 = 1101$

したがって、4 ビットでは 1101 が -3 を表す。

手順

反転して 1 を足す

6.4 ビットで見る正の数と負の数

2 進数	値
0011	3
0010	2
0001	1
0000	0
1111	-1
1110	-2
1101	-3

- ・ 何ビットで考えるかを固定することが大切
- ・ 今回は 4 ビットの例で考える

7.4 ビットで見る正の数と負の数の対応

2 進数	値	2 進数	値
0000	0	1111	-1
0001	1	1110	-2
0010	2	1101	-3
0011	3	1100	-4
0100	4	1011	-5

- ・ 正の数と負の数が対応していることが分かる
- ・ 4 ビットでは、表せる範囲に限界がある

8. ミニ実習①：補数（紙ベース）

1 2 の補数を作る

4 ビットで、次の数の 2 の補数を求めなさい。

1. 0011
2. 0001
3. 0101

2 負の数を読む

4 ビットで、次の値を答えなさい。

1. 1111
2. 1110
3. 1101

ヒント：表や「反転して1を足す」を使って考える。

9. ミニ実習①の追加：補数の対応を確認する（紙ベース）

補数の対応表を埋めよう

次の表の空欄を埋めなさい。

4 ビット表現	値
0000	0
0001	1
0010	2
1111	
1110	
1101	

ヒント：上の表や「反転して1を足す」を使って考える。

10. シフト演算とは何か

シフトとは、ビット列を左右にずらす操作である。

主なシフト：

- ・ 左シフト：左へずらす
- ・ 右シフト：右へずらす

例：

`0011 << 1 → 0110`

`0110 >> 1 → 0011`

11. 左シフトの基本

4 ビットで考える。

$0011 \ll 1 = 0110$

$0011 \ll 2 = 1100$

- ・ 左へ1つずらす
- ・ 右端には 0 が入る
- ・ 左端からはみ出したビットは落ちる

見方によっては、2 倍に近い動きになる。

12. 右シフトの基本

4 ビットで考える。

`0110 >> 1 = 0011`

`1000 >> 1 = 0100`

- ・ 右へ1つずらす
- ・ 左端には0が入る（今回はその形で考える）
- ・ 右端からはみ出したビットは落ちる

見方によっては、**1/2 に近い動き**になる。

13. シフトで何が起きるか

元の値	操作	結果
0011	<< 1	0110
0101	<< 1	1010
1000	>> 1	0100
0110	>> 1	0011

注意

シフトは見方によって2倍・1/2倍に見えるが、桁あふれが起きると単純ではなくなる。

14. シフトを比べてみよう

元の値	操作	結果
0011	<< 1	0110
0011	<< 2	1100
0011	>> 1	0001
0011	>> 2	0000

- ・ 左シフトでは、左へ動いて右に 0 が入る
- ・ 右シフトでは、右へ動いて左に 0 が入る
- ・ 動きによっては、値が大きくなったり小さくなったりする

15. ミニ実習②：シフト（紙ベース）

1 基本のシフト

4 ビットで、次の結果を求めなさい。

1. $0011 \ll 1$
2. $0101 \ll 1$
3. $1000 \gg 1$
4. $0110 \gg 1$

2 変化を読む

次の問いに答えなさい。

1. 左シフトでは、どこに 0 が入りますか。
2. 右シフトでは、どのビットが落ちますか。
3. 1111 を左に 1 つシフトすると、4 ビットではどうなりますか。

16. ミニ実習②の追加：シフトとあふれを確認する（紙ベース）

1 シフトの比較

4 ビットで、次の結果を求めなさい。

1. $0011 \ll 2$
2. $0011 \gg 2$
3. $0111 \ll 1$

2 あふれを考える

4 ビットで、次の結果を求めなさい。

1. $1111 \ll 1$
2. 左端から落ちたビットはどうなりますか。

17. オーバーフローとは何か

オーバーフローとは、表せる範囲を超えてしまうことをいう。

8 ビットで最大の値を考えると：

$$11111111_2 = FF_{16}$$

これに 1 を足すと：

$$11111111 + 1 = 1\,00000000$$

しかし、8 ビットしか使えないなら、左端の 1 は入らない。

$$00000000$$

18. オーバーフローをもう少し見てみよう

8 ビットで、次の計算を考える。

$$11111110 + 1 = 11111111$$

$$11111111 + 1 = 100000000$$

8 ビットしか使えないなら、左端の 1 は入らないので、

00000000

となる。

ポイント

表せる範囲を超えると、あふれてしまう。これを **オーバーフロー** という。

19. 浮動小数点には誤差があることがある

整数は比較的そのまま表しやすいが、**小数**は2進数でぴったり表せないことがある。

そのため、コンピュータでは

- ・ 小数の計算で少しずれることがある
- ・ 見た目どおりの値にならないことがある

今日の段階では

『小数は整数より複雑で、誤差が出ることがある』と分かれば十分。

20. まとめ

- ・ コンピュータでは、負の数も 0/1 で表す必要がある
- ・ 2 の補数は、**反転して 1 を足す** で作れる
- ・ シフトは、ビット列を左右にずらす操作である
- ・ 左シフト・右シフトでは、はみ出したビットが落ちる
- ・ ビット数には限界があり、オーバーフローが起こることがある
- ・ 小数は整数より複雑で、誤差が出ることがある

次回につながること

次回は、アルゴリズムの表し方（フローチャート・Excel 手順表）へ進む。